Plano incremental para operação real

Atualizado em: 11/11/2026 16:46 (`America/Sao_Paulo`)

Checkpoint mais recente: `2b66658` — UI-0.2 concluída

Tarefas pendentes: 69

Próxima tarefa: 6 de 74 — UI-0.3: remover fallback silencioso no detalhe do monitor

Priorização extraordinária concluída: homologação operacional de cercas e filtro de incidentes de cerca, solicitada pelo proprietário. Ela não reduz as 69 tarefas pendentes porque as tarefas 3.10–3.12 já constavam como concluídas; esta rodada comprovou o fluxo com dados persistentes de demonstração e corrigiu sua descoberta na fila.

1. Como este documento será usado

Este é o registro único e incremental do trabalho restante. A cada tarefa:

1. antes de começar, anunciar `Próxima etapa X de Y — <ID e nome>`;
2. manter a descrição e o resultado esperado abaixo;
3. ao terminar, registrar resultado obtido, testes, evidências, limitações e commit;
4. marcar a linha como concluída e reduzir a quantidade restante;
5. regenerar `PLANO_INCREMENTAL_OPERACAO_REAL.pdf` a partir deste Markdown;
6. fazer commit e push somente depois da validação proporcional ao risco.

`Resultado obtido: Pendente` nunca significa “presumidamente funcionando”. Uma tarefa só é concluída com evidência executável ou inspeção documentada. Falhas e desvios também serão registrados, sem substituir o resultado esperado.

2. Estado inicial comprovado

Entrega	Resultado esperado	Resultado obtido / evidências
UI-3.9 — agenda e permanência	Aplicar vigência, fuso, agenda semanal/ overnight e permanência sem duplicidade.	**Concluída.** Commit `2cdd6d2`; 220 testes backend, 51 frontend, integração MariaDB Tracker/Dirija Bem, TypeScript e build aprovados. Evidências detalhadas em `VALIDACOES_ETAPA_3.md`.
Homologação de cercas — rodada extraordinária	Provar círculo e polígono persistidos, atribuídos, avaliados e correlacionados a incidentes; tornar os casos encontráveis na UI.	**Concluída em 11/11/2026 16:46 -03.** Círculo permitido/SIM-1008 abriu incidente `#3516` por saída; polígono proibido/SIM-1009 abriu `#3519` por entrada. API e UI ganharam filtro por `incident_type`. Evidências: 242 backend, 65 frontend, TypeScript, build e Playwright sem erros. Documento: `docs/VALIDACAO_CERCAS_OPERACIONAIS_2026-11-11.md`. **Checkpoint:** este commit.

3. Ordem executiva

Faixa	Bloco	Quantidade	Objetivo de aceite
1–3	Concluir cercas eletrônicas	3	Cruzamento vira incidente, pode ser reproduzido e aparece ao vivo.
4–11	Verdade e segurança da interface	8	Mock, simulação, replay e produção nunca são confundidos.
12–23	Sensores e segurança patrimonial	12	Capacidade, saúde, política e comandos ficam explícitos e seguros.
24–38	Runtime dos agentes de IA	15	Agente decide com contexto real, política, evidência e auditoria.
39–52	Ações e comunicação	14	Mensagem/voz percorre fila idempotente, com recibo e limites.
53–62	Rotas, análises e gestão	10	Desvio, tendência, score e relatórios são rastreáveis.
63–74	Produção, governança e UX	12	Acesso, isolamento, observabilidade, contingência e piloto aprovados.

4. Registro das 74 tarefas pendentes

A. Concluir cercas eletrônicas

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
1/74	3.10	Integrar episódios de cerca aos incidentes.	Entrada, saída e permanência criam/correlacionam incidente operacional idempotente, preservando cerca, versão geométrica, alvo, fonte e evidência; testes de repetição e concorrência passam.	**Concluída em 10/11/2026 12:38 -03. Resultado obtido:** cada ciclo possui `violation_episode_id`; cruzamento, dwell e recuperação convergem para um incidente crítico, com vínculo direto e timeline idempotente. Falha transitória mantém o evento em fila retomável. **Evidências:** 225 testes backend; 10 integrações MariaDB aprovadas e 1 opt-in pulada; duas threads geraram um incidente/uma evidência; migrations 021/022 aplicadas; zero migrations e zero resíduos. **Limitação controlada:** evento de motorista sem veículo contextual fica `deferred`, reavaliado a cada hora e sem fabricar identidade. **Artefatos:** migrations 021/022, motor, correlacionador, worker e testes. **Checkpoint:** `78090e7`.
2/74	3.11	Criar replay de teste de cruzamento.	Replay determinístico cruza círculo e polígono nas duas direções, testa borda/agenda/dwell e produz exatamente os episódios esperados sem resíduos.	**Concluída em 10/11/2026 16:40 -03. Resultado obtido:** três cenários com relógio fixo cobrem círculo permitido, polígono proibido e agenda; borda neutra, confirmação dupla, entrada, saída, dwell e recuperação têm sequência exata. A repetição integral usa os mesmos IDs e não acrescenta registros. **Evidências:** 229 testes backend; 11 integrações MariaDB aprovadas e 1 opt-in pulada; incidentes/timelines e contagens conferidos; limpeza explícita de 10 tabelas, recibos, identidades e veículo temporário; zero resíduos. **Limitação controlada:** o replay fixa `GEOFENCE_CONFIRMATION_SAMPLES=2`, falhando explicitamente se o ambiente divergir. **Artefatos:** `geofence_replay.py` e testes unitário/MariaDB. **Checkpoint:** `b480a6d`.
3/74	3.12	Mostrar cercas e estado no mapa ao vivo.	Mapa exibe desenho, regra, alvo, estado dentro/fora, agenda, última avaliação e episódio ativo com atualização ao vivo e estados de erro/vazio.	**Concluída em 10/11/2026 16:55 -03. Resultado obtido:** endpoint agregado `geofence-live-state.v1` consulta cercas, atribuições, agendas e último estado sem N+1; o mapa desenha círculo/polígono, posição avaliada, regra, alvo, dentro/fora, agenda, avaliação e episódio, atualizando a cada 5 s e preservando o último snapshot em falha. **Evidências:** 234 backend, 54 frontend e 11 integrações MariaDB aprovados; TypeScript e build aprovados; Playwright real validou vazio e cenário círculo/polígono após dois ciclos sem duplicar paths e sem erro de página. **Limitação controlada:** snapshot limitado a 200 cercas; paginação, backoff e indicador temporal de leitura antiga permanecem como endurecimento futuro. **Artefatos:** `geofence_live_db.py`, rota `/api/geofences/live-state`, `GeofenceLiveLayer` e testes. **Checkpoint:** `e2ffb2d`.

B. Verdade e segurança da interface

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
4/74	0.1	Criar indicador global de ambiente.	Todas as telas mostram claramente produção, simulação ou replay a partir de fonte confiável; teste cobre cada modo.	**Concluída em 10/11/2026 17:28 -03. Resultado obtido:** o backend em execução virou a única fonte do selo global por meio de `runtime-environment.v1`; o frontend deixou de confiar no valor compilado. Produção exige modo global `production` e Tracker/Dirija Bem confirmados como `live`; conflito vira `MISTO`, ausência, erro, timestamp inválido ou snapshot com mais de 30 s vira `NÃO CONFIRMADO`. O contrato expõe as duas fontes, autoridade, problemas e bloqueio de ação real, sem cache, com polling a cada 15 s e sem sobreposição. **Evidências:** 240 backend e 64 frontend aprovados; TypeScript e build aprovados; matriz cobre produção, simulação, replay, misto, inválido, ausente, indisponível e vencido. Endpoint real confirmou simulação nas duas fontes; Playwright percorreu as seis abas mantendo o selo global, tooltip por fonte e zero erro de página. **Limitação controlada:** o modo é configuração declarada do processo; proveniência por pacote e bloqueio efetivo dos canais externos pertencem às etapas de sensores/ações. **Artefatos:** `runtime_environment.py`, `/api/runtime/environment`, tipos, serviço e `OperationModeBadge`. **Checkpoint:** `6cb3b59`.
5/74	0.2	Eliminar seções alimentadas por mock.	Nenhum componente de produção importa dados locais inventados; monitores, eventos, catálogo e estatísticas vêm das APIs e dos bancos alimentados pelos simuladores Tracker/Dirija Bem. Teste de fronteira impede regressão.	**Concluída em 10/11/2026 17:44 -03. Resultado obtido:** a orientação anterior de apenas etiquetar foi rejeitada pelo proprietário e substituída por remoção integral. Foram apagadas 863 linhas de `mockData`, o catálogo local de 20 capacidades não implementadas e todos os eventos/veículos inventados. Detalhe de monitor passou a consultar atribuições e eventos persistidos por veículo; catálogo/contagens vêm de `/api/events/catalog` e `/api/events/stats`. A estatística deixou de cair numa consulta pesada por erro de import e usa eventos restaurados/persistidos mais o catálogo do banco. **Evidências:** busca no código de produção encontra zero import de `mockData`; teste de fronteira bloqueia sua reintrodução; 241 backend e 65 frontend aprovados, TypeScript e build aprovados. API real mostrou quatro tipos suportados e 70 eventos do processo; Playwright exibiu catálogo e 200 eventos persistidos do Monitor #1, sem rótulo de demonstração e sem erro de página. **Decisão de dados:** não foi necessário fabricar carga extra; os dois simuladores continuam alimentando Tracker e Dirija Bem, e o volume existente já permite demonstrar. **Limitação controlada:** os monitores atualmente atribuídos no banco são Tracker; integrar alvos Dirija Bem aos monitores exige configuração operacional, não dados locais falsos. **Checkpoint:** `2b66658`.
6/74	0.3	Remover fallback silencioso no detalhe do monitor.	Falha de API gera erro explícito; nenhum score ou veículo inventado aparece como real.	**Pendente.**
7/74	0.4	Criar estado comum de erro, vazio e carregamento.	Componentes reutilizam contrato visual/acessível e testes cobrem os três estados.	**Pendente.**
8/74	0.5	Centralizar base URL e cliente HTTP.	App 2 faz chamadas somente pelo cliente configurável; busca estática não encontra URLs locais espalhadas.	**Pendente.**
9/74	0.6	Impedir URL hardcoded em componentes novos.	Teste automatizado falha ao introduzir `localhost`, IP ou URL absoluta fora da configuração permitida.	**Pendente.**
10/74	0.7	Documentar matriz tela → endpoint → fonte → modo.	Cada bloco visual aponta endpoint, banco/fonte, modo e comportamento em erro; matriz é conferida contra o código.	**Pendente.**

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
11/74	0.8	Separar Tracker e Dirija Bem mantendo visão unificada.	Origem fica inequívoca em lista, detalhe, eventos e relatórios, com filtros e total unificado corretos.	**Pendente.**

C. Sensores e segurança patrimonial

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
12/74	4.1	Catalogar capacidades por protocolo/ dispositivo.	Catálogo versionado informa o que cada equipamento realmente mede/comanda, sem prometer recurso inexistente.	**Pendente.**
13/74	4.2	Criar modelo de sensor e leitura.	Leituras guardam sensor, valor, unidade, qualidade, fonte, horário da fonte/recebimento e correlação.	**Pendente.**
14/74	4.3	Criar modelo de política de detector.	Política separa capacidade física de regra operacional, com versão, escopo, limites e auditoria.	**Pendente.**
15/74	4.4	Criar aba Sensores do veículo.	Detalhe apresenta sensores suportados/configurados sem mock e com estados acessíveis.	**Pendente.**
16/74	4.5	Exibir valor, saúde, origem e última leitura.	Operador distingue leitura atual, desatualizada, inválida ou indisponível e rastreia a fonte.	**Pendente.**
17/74	4.6	Ativar política, não hardware inexistente.	A interface deixa explícito que habilita detecção; comando físico só aparece quando suportado e confirmado.	**Pendente.**
18/74	4.7	Criar estados armado/desarmado.	Estado possui transições autorizadas, responsável, horário, escopo e histórico auditável.	**Pendente.**
19/74	4.8	Detectar alimentação removida e bateria baixa.	Apenas protocolos com sinal comprovado geram episódio, com histerese/cooldown e testes de falso positivo.	**Pendente.**
20/74	4.9	Detectar offline contextualizado.	Offline considera frequência esperada, viagem, ignição, manutenção e indisponibilidade da integração.	**Pendente.**
21/74	4.10	Definir contrato de comando e confirmação.	Comando possui solicitação, envio, ACK/NACK/timeout, idempotência, estado final e correlação.	**Pendente.**
22/74	4.11	Reforçar autorização para comandos críticos.	Bloqueio/acionamento crítico exige papel, confirmação, motivo, política e trilha imutável.	**Pendente.**
23/74	4.12	Criar simulador de sinais suportados.	Simulador gera somente sinais documentados e cenários determinísticos de sucesso/falha para Tracker.	**Pendente.**

D. Runtime dos agentes de IA

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
24/74	5.1	Versionar configuração do agente.	Migration guarda objetivo, instruções, modelo, política, limites, status e versão imutável.	**Pendente.**
25/74	5.2	Persistir execuções e decisões.	Cada execução registra entrada, contexto, versão, saída, latência, custo, erro e correlação.	**Pendente.**
26/74	5.3	Definir decisão estruturada da IA.	Schema validável contém classificação, confiança, evidências, risco, ações propostas e necessidade humana.	**Pendente.**
27/74	5.4	Implementar provedor de IA por interface.	Runtime troca provedor/modelo por configuração, sem acoplar regra operacional ao SDK externo.	**Pendente.**

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
28/74	5.5	Implementar timeout, fallback e circuit breaker.	Falha/latência da IA não para monitoramento; fallback é explícito, medido e testado.	**Pendente.**
29/74	5.6	Criar ferramentas somente leitura.	Agente consulta veículo, posição, histórico e cerca com escopo, limite e retorno estruturado auditável.	**Pendente.**
30/74	5.7	Criar política de ferramentas por agente.	Allowlist por versão bloqueia ferramenta/parâmetro fora da função do agente; testes provam negação.	**Pendente.**
31/74	5.8	Criar níveis de autonomia.	Níveis separam observar, recomendar, solicitar aprovação e executar; risco máximo limita autonomia.	**Pendente.**
32/74	5.9	Criar worker de análise de incidente.	Worker idempotente coleta contexto, chama decisão, persiste resultado e respeita fila/prioridade.	**Pendente.**
33/74	5.10	Persistir evidências, decisão e confiança.	Operador consegue reconstruir por que o agente decidiu, com dados e versões usadas.	**Pendente.**
34/74	5.11	Criar tela de configuração do agente.	UI edita objetivo, modelo, limites, ferramentas e autonomia com validação/versionamento.	**Pendente.**
35/74	5.12	Criar tela de execuções e decisões.	Timeline mostra entrada, evidência, raciocínio estruturado permitido, decisão, custo, duração e falha.	**Pendente.**
36/74	5.13	Implementar pausar e assumir manualmente.	Humano pausa agente e assume incidente sem corrida; estado e autoria ficam auditados.	**Pendente.**
37/74	5.14	Criar modo simulação sem ações externas.	Agente percorre fluxo completo e registra ações que faria, mas nenhum canal externo é acionado.	**Pendente.**
38/74	5.15	Avaliar com casos esperados.	Dataset cobre risco, normalidade, falso positivo/negativo e mede precisão, segurança e consistência.	**Pendente.**

E. Ações e comunicação

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
39/74	6.1	Criar migration da fila de ações.	Ação guarda tipo, alvo, payload, política, prioridade, estado, tentativas e idempotency key.	**Pendente.**
40/74	6.2	Implementar máquina de estados e idempotência.	Transições válidas são atômicas; retry não duplica mensagem, ligação ou comando.	**Pendente.**
41/74	6.3	Criar notificação interna.	Primeira ferramenta gera notificação persistente, recebida na UI e ligada ao incidente.	**Pendente.**
42/74	6.4	Integrar mensagem em sandbox.	Canal envia apenas para destino de teste, com credencial externa ao código e recibo real.	**Pendente.**
43/74	6.5	Persistir entrega, leitura e resposta.	Status do provedor é normalizado e aparece na timeline com timestamps e payload sanitizado.	**Pendente.**
44/74	6.6	Criar templates versionados.	Mensagem usa template aprovado, variáveis tipadas, idioma, versão e prévia.	**Pendente.**
45/74	6.7	Criar contatos e escalonamento.	Proprietário/gestor/central têm consentimento, canais, prioridade e ordem de contato auditáveis.	**Pendente.**
46/74	6.8	Criar solicitação/aprovação humana.	Ação acima da autonomia aguarda aprovação explícita com expiração, motivo e versão.	**Pendente.**
47/74	6.9	Criar timeline de ações no incidente.	Tentativa, aprovação, envio, entrega, resposta e falha aparecem em ordem e com correlação.	**Pendente.**
48/74	6.10	Implementar retentativas e fila de falhas.	Backoff, limite, dead-letter e reprocessamento manual impedem perda ou loop infinito.	**Pendente.**
49/74	6.11	Integrar voz em sandbox.	Ligação de teste usa roteiro/versionamento, identifica automação e persiste recibo.	**Pendente.**
50/74	6.12	Transcrever e classificar resposta.	Áudio autorizado vira transcrição/classificação estruturada com confiança e revisão humana.	**Pendente.**

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
51/74	6.13	Limitar tentativa, horário e custo.	Política impede assédio, contato fora do horário, custo excessivo e repetição sem progresso.	**Pendente.**
52/74	6.14	Testar opt-out, falha e ausência de resposta.	Casos de recusa, número inválido, provedor fora, silêncio e timeout terminam em estado seguro.	**Pendente.**

F. Rotas, análises e gestão

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
53/74	7.1	Criar CRUD de rotas e corredor.	Rota/polilinha versionada possui tolerância, pontos, autoria e validação geométrica.	**Pendente.**
54/74	7.2	Criar atribuição e janela de execução.	Veículo/motorista recebe rota com vigência, fuso, início/fim e estado da execução.	**Pendente.**
55/74	7.3	Detectar desvio e parada não planejada.	Episódios usam tolerância/histerese/permanência e evitam alertas por ruído ou parada prevista.	**Pendente.**
56/74	7.4	Criar replay de viagem.	Operador reproduz posições/eventos/rota na ordem temporal, com velocidade controlável e teste determinístico.	**Pendente.**
57/74	7.5	Adicionar período e comparação às análises.	UI/API comparam períodos equivalentes, mostram filtros e evitam misturar fontes/janelas.	**Pendente.**
58/74	7.6	Normalizar comportamento por distância/tempo.	Métricas usam exposição (km/hora de condução), com fórmula e dados ausentes explícitos.	**Pendente.**
59/74	7.7	Criar relatórios por Tracker e Dirija Bem.	Relatório separado/unificado mantém origem, definição das métricas e rastreabilidade.	**Pendente.**
60/74	7.8	Criar indicadores de SLA e automação.	Painel mede detecção, atendimento, resolução, ação automática, intervenção e falha.	**Pendente.**
61/74	7.9	Criar exportação e agendamento.	Exportação respeita filtros/permissões e agendamento registra geração, entrega e erro.	**Pendente.**
62/74	7.10	Validar score com amostra conhecida.	Dataset rotulado prova fórmula/faixas e documenta erro, viés, falso positivo e falso negativo.	**Pendente.**

G. Produção, governança e UX

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
63/74	8.1	Implementar autenticação.	Login/sessão seguros, expiração, recuperação e proteção de endpoints/UI são testados.	**Pendente.**
64/74	8.2	Implementar papéis e permissões.	Menor privilégio controla leitura, operação, configuração, aprovação e comando crítico.	**Pendente.**
65/74	8.3	Implementar isolamento por empresa.	Toda consulta/mutação aplica tenant; testes tentam acesso cruzado e falham.	**Pendente.**
66/74	8.4	Criar auditoria consultável.	Mudanças críticas possuem ator, antes/depois, motivo, correlação e consulta protegida.	**Pendente.**
67/74	8.5	Criar painel de integrações e saúde.	Tracker, Dirija Bem, IA e canais mostram disponibilidade, atraso, fila e último sucesso/erro.	**Pendente.**
68/74	8.6	Adicionar métricas, logs e alertas técnicos.	Sinais estruturados permitem detectar ingestão parada, erro, latência, fila e custo anormal.	**Pendente.**
69/74	8.7	Executar teste de carga e recuperação.	Limites são medidos; queda/restart não perde nem duplica episódio, incidente ou ação.	**Pendente.**
70/74	8.8	Auditar segurança e privacidade.	Credenciais, dados pessoais, retenção, consentimento, criptografia e dependências têm achados tratados.	**Pendente.**

Ordem	ID	Descrição	Resultado esperado / prova de aceite	Resultado obtido
71/74	8.9	Executar teste de acessibilidade.	Fluxos críticos passam teclado, foco, contraste, rótulos e leitor de tela conforme meta definida.	**Pendente.**
72/74	8.10	Executar teste operacional com playbooks.	Operadores executam incidentes normais/críticos, falhas e takeover com critérios aprovados.	**Pendente.**
73/74	8.11	Publicar runbook e contingência.	Documento cobre subir/parar, diagnóstico, rollback, indisponibilidade e contatos responsáveis.	**Pendente.**
74/74	8.12	Liberar produção por grupo piloto.	Piloto limitado possui métricas, consentimento, suporte, rollback, critérios de sucesso e aprovação formal.	**Pendente.**

5. Formato obrigatório ao concluir uma tarefa

Para cada linha concluída, substituir `Pendente` por um registro com:

- **Resultado obtido:** comportamento realmente entregue;
- **Evidências:** testes, endpoints, consultas, Playwright, métricas e ausência de resíduos;
- **Desvios/limitações:** diferença entre esperado e obtido, sem ocultar risco;
- **Artefatos:** migrations, APIs, telas e documentos alterados;
- **Checkpoint:** hash do commit publicado;
- **Data/hora:** no fuso `America/Sao_Paulo`.

6. Próxima execução

Próxima etapa 6 de 74 — UI-0.3: remover fallback silencioso no detalhe do monitor.

Ao concluí-la, este documento deverá indicar `68 tarefas pendentes` e anunciar `Próxima etapa 7 de 74 — UI-0.4`, preservando o denominador original para mostrar o avanço total da rodada.